

02. JS Type Conversion and Coercion

Complete Guide to Type Conversion and Coercion in JavaScript

Interactive Lab Tutorial

Setup: VS Code + Live Server

What you'll learn: How JavaScript automatically converts types (coercion) and how you can explicitly convert types (conversion) — with hands-on examples you'll build step by step.

Initial Setup

Step 1: Create your project structure

```
js-types-lab/  
├─ index.html  
└─ script.js
```

Step 2: Create `index.html`

```
<!DOCTYPE DOCTYPE html>  
<html lang="en">  
  <head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>JS Type Conversion Lab</title>  
  </head>  
  <body>  
    <h1>🔗 JavaScript Type Conversion & Coercion Lab</h1>  
    <div id="output" class="output">  
      <p>Check the console (F12) for detailed outputs!</p>  
    </div>  
  
    <script src="script.js"></script>  
  </body>  
</html>
```

Step 3: Create an empty `script.js` file

Step 4: Right-click on `index.html` → Open with Live Server

Part 1: Understanding JavaScript Types

Concept: JavaScript has 7 primitive types and 1 complex type. Let's see them all!

Add to `script.js`:

```
console.log("=== PART 1: JavaScript Types ===\n");

// Primitive Types
let str = "Hello";
let num = 42;
let bool = true;
let undef = undefined;
let nul = null;
let sym = Symbol("unique");
let bigInt = 123456789012345678901234567890n;

// Complex Type
let obj = { name: "John" };

// Check types using typeof
console.log("String:", typeof str);      // "string"
console.log("Number:", typeof num);      // "number"
console.log("Boolean:", typeof bool);    // "boolean"
console.log("Undefined:", typeof undef); // "undefined"
console.log("Null:", typeof nul);        // "object" (this is a JS bug!)
console.log("Symbol:", typeof sym);      // "symbol"
console.log("BigInt:", typeof bigInt);   // "bigint"
console.log("Object:", typeof obj);      // "object"
```

What to observe: Check your console. Notice that `typeof null` returns "object" — this is a famous JavaScript quirk!

Part 2: Type Coercion (Automatic Conversion)

Concept: JavaScript automatically converts types when needed. This can be helpful or surprising!

2.1: String Coercion

Add this to your script:

```

console.log("\n=== PART 2.1: String Coercion ===\n");

// When you use + with a string, everything becomes a string
console.log("5" + 3);           // "53" (number 3 becomes string "3")
console.log("Hello" + true);    // "Hellotrue"
console.log("Value: " + null);  // "Value: null"
console.log("Count: " + undefined); // "Count: undefined"

// Multiple values
console.log("The answer is " + 20 + 22); // "The answer is 2022" (left to right)
console.log(20 + 22 + " is the answer"); // "42 is the answer" (numbers add first!)

```

💡 **Key Learning:** The `+` operator with strings acts like glue — everything sticks together as text!

2.2: Number Coercion

Add this next:

```

console.log("\n=== PART 2.2: Number Coercion ===\n");

// All other math operators convert to numbers
console.log("10" - 5);          // 5 (string "10" becomes number 10)
console.log("10" * "2");        // 20 (both strings become numbers)
console.log("20" / "4");        // 5
console.log("10" % 3);          // 1 (modulo still works)

// Tricky cases
console.log("5" - true);        // 4 (true becomes 1)
console.log("5" - false);       // 5 (false becomes 0)
console.log("10" - "abc");      // NaN (can't convert "abc" to number)
console.log("10" * null);       // 0 (null becomes 0)
console.log("10" - undefined);  // NaN (undefined can't become a number)

```

💡 **Key Learning:** Minus, multiply, divide convert to numbers. Only plus (+) is special with strings!

2.3: Boolean Coercion

Add this:

```

console.log("\n=== PART 2.3: Boolean Coercion ===\n");

// Falsy values (become false)
console.log("Boolean(0):", Boolean(0));           // false
console.log("Boolean(''):", Boolean(''));         // false (empty string)
console.log("Boolean(null):", Boolean(null));      // false
console.log("Boolean(undefined):", Boolean(undefined)); // false
console.log("Boolean(NaN):", Boolean(NaN));        // false

// Truthy values (become true)
console.log("Boolean(1):", Boolean(1));           // true
console.log("Boolean('hello'):", Boolean('hello')); // true
console.log("Boolean(' '):", Boolean(' '));       // true (space is not empty!)
console.log("Boolean([]):", Boolean([]));         // true (empty array is
truthy!)
console.log("Boolean({}):", Boolean({}));         // true (empty object is
truthy!)

// Practical use in conditions
let userName = "";
if (userName) {
    console.log("Welcome, " + userName);
} else {
    console.log("Empty string is falsy, so this runs!");
}

```

💡 **Key Learning:** Remember the 5 falsy values: `0`, `""`, `null`, `undefined`, `NaN`. Everything else is truthy!

2.4: Equality and Coercion

Add this important section:

```
console.log("\n=== PART 2.4: Equality Coercion ===\n");

// == (loose equality) does type coercion
console.log("5 == '5':", 5 == '5');      // true (string converted to number)
console.log("0 == false:", 0 == false);  // true
console.log("' ' == false:", ' ' == false); // true
console.log("null == undefined:", null == undefined); // true

// === (strict equality) does NOT coerce
console.log("5 === '5':", 5 === '5');    // false (different types)
console.log("0 === false:", 0 === false); // false
console.log("null === undefined:", null === undefined); // false

// The recommendation: always use ===
console.log("\n⚠️ Best Practice: Use === to avoid surprises!");
```

💡 **Key Learning:** Use `===` (strict equality) to avoid unexpected type conversions!

Part 3: Explicit Type Conversion

Concept: You can manually convert types when you know what you're doing.

3.1: Converting to String

Add this:

```
console.log("\n≡ PART 3.1: Converting to String ≡\n");

let num1 = 42;
let bool1 = true;
let arr1 = [1, 2, 3];

// Method 1: String() function
console.log("String(42):", String(num1));    // "42"
console.log("String(true):", String(bool1)); // "true"
console.log("String(null):", String(null));  // "null"

// Method 2: .toString() method
console.log("(42).toString():", num1.toString()); // "42"
console.log("(true).toString():", bool1.toString()); // "true"
console.log("[1,2,3].toString():", arr1.toString()); // "1,2,3"

// Method 3: Template literals (modern way)
console.log(`Value: ${num1}`); // "Value: 42"

// Special: Converting with different bases
console.log("(255).toString(16):", (255).toString(16)); // "ff" (hexadecimal)
console.log("(8).toString(2):", (8).toString(2));        // "1000" (binary)
```

3.2: Converting to Number

Add this:

```
console.log("\n≡ PART 3.2: Converting to Number ≡\n");

// Method 1: Number() function
console.log("Number('42'):", Number('42')); // 42
console.log("Number('3.14'):", Number('3.14')); // 3.14
console.log("Number(''):", Number('')); // 0 (empty string)
console.log("Number('hello'):", Number('hello')); // NaN
console.log("Number(true):", Number(true)); // 1
console.log("Number(false):", Number(false)); // 0
console.log("Number(null):", Number(null)); // 0
console.log("Number(undefined):", Number(undefined)); // NaN

// Method 2: parseInt() and parseFloat()
console.log("parseInt('42px'):", parseInt('42px')); // 42 (stops at first non-
digit)
console.log("parseInt('100', 2):", parseInt('100', 2)); // 4 (binary to decimal)
console.log("parseFloat('3.14.15'):", parseFloat('3.14.15')); // 3.14

// Method 3: Unary + operator (quick trick!)
console.log("+'42':", +'42'); // 42
console.log("+'true':", +'true'); // 1
console.log("+'3.14':", +'3.14'); // 3.14

// Practical example
let userInput = "25";
let age = Number(userInput);
console.log("\nUser entered:", userInput, "(type:", typeof userInput, ")");
console.log("Converted to:", age, "(type:", typeof age, ")");
```

3.3: Converting to Boolean

Add this:

```
console.log("\n=== PART 3.3: Converting to Boolean ===\n");

// Method 1: Boolean() function
console.log("Boolean(1):", Boolean(1));           // true
console.log("Boolean(0):", Boolean(0));           // false
console.log("Boolean('text'):", Boolean('text')); // true
console.log("Boolean(''):", Boolean(''));         // false

// Method 2: Double NOT operator !! (common trick)
console.log("!!'text':", !!'text');              // true
console.log("!!0:", !!0);                         // false
console.log("!!null:", !!null);                   // false
console.log("!![]:", !![]);                       // true

// Practical use: checking if value exists
let username = "Alice";
let hasUsername = !!username;
console.log("\nHas username?", hasUsername);
```

Part 4: Common Pitfalls and Gotchas

Add this important section:


```
console.log("\n=== PART 4: Common Pitfalls ===\n");

// Pitfall 1: Array to number
console.log("Number([]):", Number([]));      // 0 (empty array)
console.log("Number([5]):", Number([5]));    // 5 (single element)
console.log("Number([1,2]):", Number([1,2])); // NaN (multiple elements)

// Pitfall 2: Object to primitive
console.log("Number({}):", Number({}));      // NaN
console.log("String({}):", String({}));      // "[object Object]"

// Pitfall 3: Adding arrays
console.log("[1,2] + [3,4]:", [1,2] + [3,4]); // "1,23,4" (converted to strings!)

// Pitfall 4: null vs undefined
console.log("Number(null):", Number(null));   // 0
console.log("Number(undefined):", Number(undefined)); // NaN

// Pitfall 5: Comparing different types
console.log("'2' > '12':", '2' > '12');      // true (string comparison!)
console.log("2 > 12:", 2 > 12);              // false (number comparison)

console.log("\n⚠️ Always be explicit about your types to avoid bugs!");
```

Part 5: Real-World Practical Examples

Add this final section:

```
console.log("\n=== PART 5: Real-World Examples ===\n");

// Example 1: Form input validation
function validateAge(input) {
  let age = Number(input);

  if (isNaN(age)) {
    return "Please enter a valid number";
  }

  if (age < 0 || age > 150) {
    return "Please enter a realistic age";
  }

  return "Age is valid: " + age;
}

console.log("Validate '25':", validateAge('25'));
console.log("Validate 'abc':", validateAge('abc'));
console.log("Validate '200':", validateAge('200'));

// Example 2: Price calculator
function calculateTotal(priceString, quantityString) {
  let price = parseFloat(priceString);
  let quantity = parseInt(quantityString);

  if (isNaN(price) || isNaN(quantity)) {
    return "Invalid input";
  }

  let total = price * quantity;
  return "$" + total.toFixed(2);
}

console.log("\nPrice calc:", calculateTotal('19.99', '3'));

// Example 3: Boolean flags from strings
function getUserPermissions(isAdminString) {
  // API might return "true" or "false" as strings
  let isAdmin = isAdminString === "true";
  return {
    canEdit: isAdmin,
    canDelete: isAdmin,
    canView: true
  };
}
```

```
console.log("\nPermissions:", getUserPermissions("true"));

// Example 4: Safe number conversion
function safeNumber(value, defaultValue = 0) {
  let num = Number(value);
  return isNaN(num) ? defaultValue : num;
}

console.log("\nSafe conversion of 'abc':", safeNumber('abc', 100));
console.log("Safe conversion of '42':", safeNumber('42', 100));
```

Part 6: Interactive Practice Section

Add this for hands-on practice:

```
console.log("\n=== PART 6: Practice Exercises ===\n");

// Exercise 1: Predict the output
console.log("Exercise 1 - Predict before running:");
console.log("'5' + 3 + 2 =", '5' + 3 + 2);
console.log("5 + 3 + '2' =", 5 + 3 + '2');
console.log("'5' - 3 + 2 =", '5' - 3 + 2);

// Exercise 2: Fix the bug
console.log("\nExercise 2 - Buggy code:");
let userScore = "95";
let bonusPoints = "5";
// Bug: This will give "955" instead of 100
console.log("Wrong:", userScore + bonusPoints);
// Fix: Convert to numbers first
console.log("Fixed:", Number(userScore) + Number(bonusPoints));

// Exercise 3: Write a function
console.log("\nExercise 3 - Your turn!");
// TODO: Write a function that takes a string and returns true if it's a valid
number
function isValidNumber(str) {
    return !isNaN(Number(str)) && str.trim() !== '';
}

console.log("isValidNumber('42'):", isValidNumber('42'));
console.log("isValidNumber('abc'):", isValidNumber('abc'));
console.log("isValidNumber(''):", isValidNumber(''));
```

Quick Reference Card

Add this summary at the end:

```
console.log("\n≡ QUICK REFERENCE ≡\n");
```

```
console.log(`
```

```
📄 QUICK REFERENCE CARD
```

TO STRING:

String(value)	→ Best for any value
value.toString()	→ Best for numbers (can specify base)
`\`\${value}\``	→ Best in template literals

TO NUMBER:

Number(value)	→ Best for strict conversion
parseInt(value)	→ Best for integers from strings
parseFloat(value)	→ Best for decimals from strings
+value	→ Quick shorthand

TO BOOLEAN:

Boolean(value)	→ Explicit conversion
!!value	→ Quick shorthand

COMPARISON:

≡	→ Always use (strict)
==	→ Avoid (loose, coerces types)

FALSY VALUES (only 6!):

false, 0, "", null, undefined, NaN

TRUTHY VALUES:

Everything else! Including [], {}, "0", "false"

```
`);
```

Your Practice Challenges

Now try these on your own! Add these challenges to your script and solve them:

```
console.log("\n=== YOUR CHALLENGES ===\n");

// Challenge 1: Create a function that safely adds two values
// It should handle strings, numbers, null, undefined
function safeAdd(a, b) {
  // Your code here
  // Hint: Use Number() and check for NaN
}

// Challenge 2: Create a function that checks if a value is "empty"
// Empty means: "", 0, null, undefined, false, NaN, [], {}
function isEmpty(value) {
  // Your code here
}

// Challenge 3: Create a currency formatter
// Input: "1234.5" → Output: "$1,234.50"
function formatCurrency(value) {
  // Your code here
  // Hint: Use Number(), toFixed(), and string methods
}

// Test your solutions
console.log("Test safeAdd:", safeAdd("5", "10"));
console.log("Test isEmpty:", isEmpty(""));
console.log("Test formatCurrency:", formatCurrency("1234.5"));
```

Summary and Best Practices

Key Takeaways:

1. **Type Coercion is automatic** — JavaScript converts types behind the scenes
2. **Type Conversion is manual** — You explicitly tell JavaScript what type you want
3. **Use `===` instead of `==`** — Avoid unexpected coercion in comparisons
4. **Know your falsy values** — Only 6: `false`, `0`, `""`, `null`, `undefined`, `NaN`
5. **Be explicit** — Use `Number()`, `String()`, `Boolean()` to make your intentions clear
6. **Validate user input** — Always convert and check data from forms/APIs

Next Steps:

- Experiment with the code
- Try the challenges
- Add `console.log()` everywhere to see what's happening

- Open DevTools (F12) and play in the Console tab
 - Break things on purpose to learn!
-

Happy coding! 🚀